ElderCare – Emergency Assist

Implemented Modules, Functionalities & Technologies

This document describes the functionality present in the current browser-based prototype and the technologies used to implement it. It also identifies important integrations that are not yet connected.

1. Implemented Modules and Functionalities

1. User Registration and Authentication

Supports elderly and caregiver roles, separate login flows, email-based accounts, SHA-256 password hashing with the Web Crypto API, password reset, and caregiver linking using the elderly user's email address.

2. Elderly Dashboard

Displays the elderly user's status, online/offline state, navigation, and browser permission status.

3. Voice-Based Emergency Detection

Uses the browser SpeechRecognition/Web Speech API to listen for configured emergency phrases. Includes English and selected Hindi/Roman-Hindi test phrases, confidence scoring, and normal/negative phrases intended to reduce false alarms. The test suite contains 12 labelled phrases.

4. Emergency Alert and SOS

A detected phrase can start a 3-second cancellation countdown and then create a simulated emergency event. The interface prepares call, SMS, and email actions through tel:, sms:, and mailto: links; it also supports alert history, caregiver response, and resolution.

5. GPS and Location Sharing

Uses browser geolocation to obtain and watch the device location, record latitude/longitude and accuracy, retain the last known position, and create Google Maps/location-sharing links.

6. Medical Records

Allows users to save and delete profile information such as name, age, blood group, allergies, medical conditions, medical history, doctor information, and emergency notes.

7. Medication Management

Supports adding/removing medicines, dosage, quantity and remaining stock, start/end dates, morning/afternoon/night schedules, marking doses as taken, low-stock indicators, medication history, test reminders, and local refill-order records.

8. Appointment Management

Allows appointments to be created, viewed, and removed with doctor, hospital/place, date, and time details; upcoming appointments can be displayed.

9. Emergency Contacts

Supports contact names, roles, phone numbers, email addresses, priority, preferred notification methods, phone links, and predefined Indian helpline entries.

10. Caregiver Dashboard

Shows linked-patient information, active alerts, location, medical summary, medication adherence and low-stock information, appointments, check-ins, and response actions.

11. Caregiver Alert Handling

Displays active alerts and alert history, and provides actions to respond, call the patient, open an ambulance-call link, view the location, and resolve an alert.

12. Check-In System

Allows a caregiver to send a check-in prompt. The elderly user can respond “I am OK” or “I need help”; the help response triggers the emergency flow.

13. Care Journal

Records entries such as blood pressure, blood sugar, weight, mood, meals, and general notes, along with caregiver and timestamp information.

14. Emergency History and Notifications

Maintains emergency event details such as detected phrase, language label, confidence, date/time, location status, call/SMS/email link-action status, and response status.

15. Demo and Testing Mode

Provides controls to simulate an emergency, test voice detection, GPS, SMS/email links, and medication reminders.

16. Performance and Test Reporting

Includes a 12-phrase test suite and reports accuracy, false positives/negatives, language-wise results, average detection time, GPS success, and counts of opened SMS/email/call links.

17. Security and Audit Features

Includes SHA-256 password hashing, role-based interface access, inactivity logout after 15 minutes, HTML input escaping, and audit logs.

18. User Interface and Browser Storage

Includes responsive layouts, light/dark theme, online/offline status, LocalStorage persistence using the key ecv2, cross-tab storage-event updates, audio alerts, speech prompts, and browser-based interactions.

2. Technologies and Browser APIs

Technology / API	Purpose in the prototype
HTML5	Page structure and semantic content.
CSS3	Styling, responsive layout, themes, and interface presentation.
Vanilla JavaScript	Application logic and browser-side interactions.
Web Speech API (SpeechRecognition)	Voice input and emergency phrase recognition, where supported by the browser.
Geolocation API	Retrieving and monitoring device location, subject to user permission.
Web Crypto API (SHA-256)	Client-side password hashing.
LocalStorage API	Saving prototype data in the browser under the key ecv2.
Web Storage storage event	Updating views across browser tabs/windows.

Technology / API	Purpose in the prototype
Speech Synthesis API	Spoken prompts and feedback.
Web Audio API	Audio alerts and sound feedback.
Google Maps URL integration	Opening a map/location link.
Google Fonts	Playfair Display and Lora typefaces.

3. Database Design

A separate SQLite database schema has been prepared with 18 tables: users, elder_profiles, caregiver_links, emergency_contacts, medications, medication_schedules, medication_logs, refill_orders, appointments, locations, emergency_events, emergency_event_timeline, emergency_contact_actions, notifications, check_ins, care_journal, audit_logs, and emergency_services.

Important: The SQLite database is currently separate from the website. The browser prototype stores its data in LocalStorage; the SQLite schema is not yet connected to a backend.

4. Current Limitations and Integration Status

- SMS, email, and phone actions open or prepare the relevant external application/link. The prototype does not automatically send messages or place calls in the background.
- Emergency detection and alert handling are browser-side prototype functions; they are not a verified connection to emergency services.
- Medication refill entries are local tracking records. There is no live pharmacy or automatic medicine-order integration.
- Data is stored locally in the browser. There is no cloud database, backend API, or automatic multi-device synchronization.
- Speech recognition depends on browser support, microphone permission, and configured recognition phrases. Script/language labelling should not be interpreted as full speech-recognition support for every Indian language.

5. Suggested Future Enhancements (Not Yet Implemented)

- Connect a Python Flask backend and the prepared SQLite database.
- Add secure cloud storage, authenticated APIs, and multi-device synchronization.
- Integrate an SMS/email provider for actual message delivery and an approved emergency-calling workflow.
- Integrate a pharmacy partner for real prescription/refill processing.
- Use stronger password hashing such as bcrypt/Argon2 and encryption for sensitive medical data.
- Expand and validate multilingual speech recognition; investigate AI/ML-based emergency detection with safety testing.

Project status: Browser-based prototype with local persistence. The database schema is prepared separately; backend and third-party service integrations remain future work.